Planeta FP

0371. FUNDAMENTOS DEL HARDWARE

NF1.1 Arquitectura de sistemas

REPRESENTACIÓN DE LA INFORMACIÓN I ARQUITECTURAS

Septiembre 2026

Marina Iriarte

RED DE EDUCACIÓN SUPERIOR
Planeta Formación y Universidades

Contenido

NF1. Arquitectura de sistemas

- **RA1.** Configura equipos microinformáticos, componentes y periféricos, analizando sus características y su relación con el conjunto.

Representación de la información y arquitecturas

1. **Representación de la información**
   - Sistemas de numeración
   - Teorema fundamental de la numeración
   - Sistema binario, octal y hexadecimal
   - Operaciones aritméticas y lógicas
   - Puertas lógicas
   - Códigos entrada/salida E/S
   - Puertas lógicas
2. **Comunicación**
   - Usuario y Ordenador
   - Direccionalidad
   - Sincronización
   - Tipo de señal
3. **Arquitectura**
   - Von Neumann
   - Harvard
   - Arquitecturas paralelas:
   - Implícito
   - Explícito
   - Multihilo

Representación de la información

Un sistema informático tiene como objetivo representar algún tipo de información para que la persona pueda utilizarla o comunicarse con otros sistemas.

Normalmente se introduce la información en el sistema con un lenguaje que se puede entender por los seres humanos basado:

- Alfabéticos (A-Z, a-z)
- Numéricos (0-9)

caracteres alfanuméricos

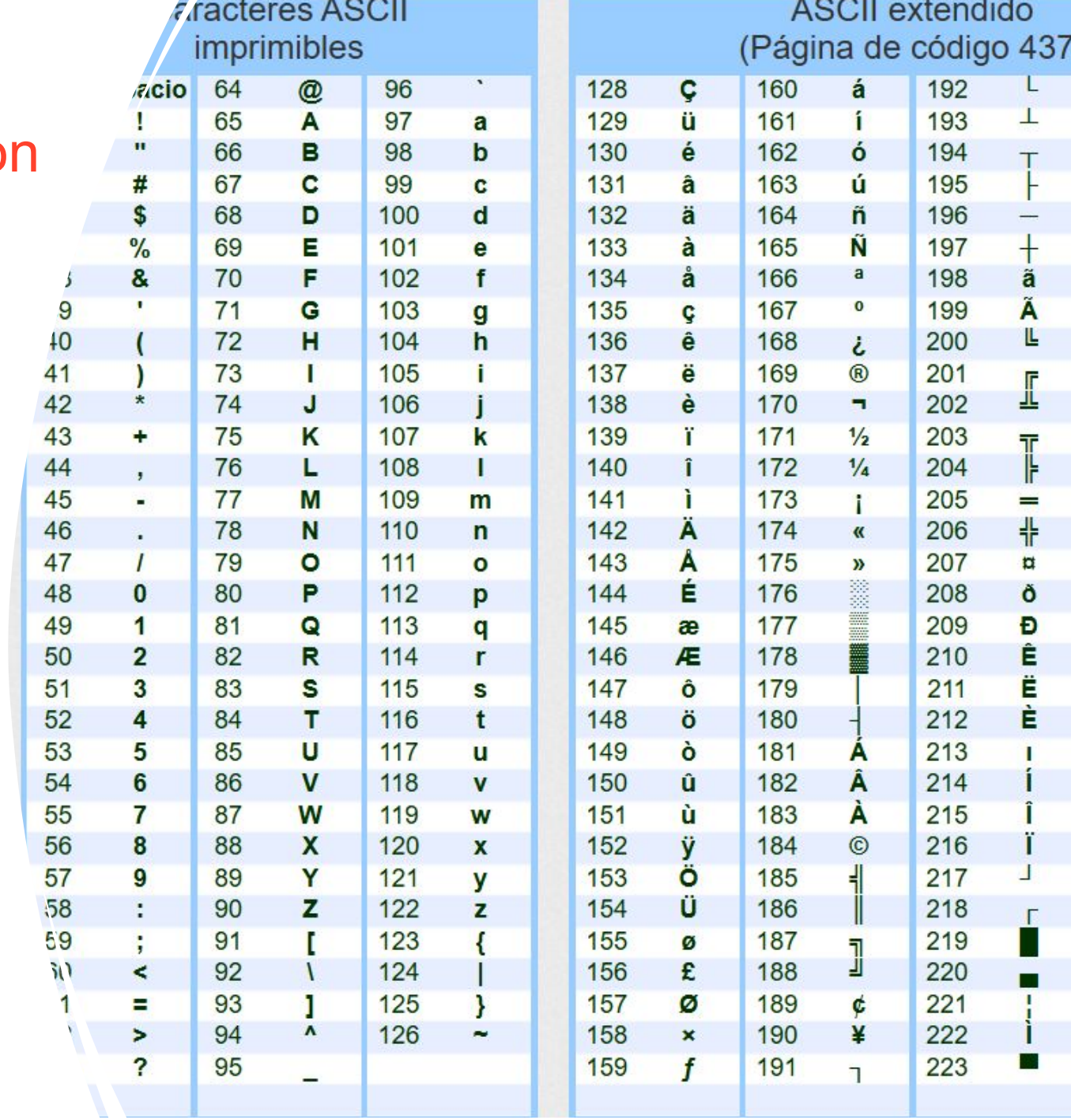
- Especiales (@, #, %, etc.)
- De control (intro, tabulador...)
- Gráficos (símbolos visuales simples)

caracteres de texto

El conjunto de símbolos que usamos (letras, números, signos...) forma lo que llamamos un **juego de caracteres *(character set)***.
Cada carácter tiene un **número asociado**, que luego se convierte en binario para que el ordenador lo pueda entender.

Representación de la información

- Ejemplos de **juegos de caracteres (character set)**:
  - **ASCII**(básico, 128 caracteres o extended 255 caracteres)
  - **ISO-8859-1** (amplía ASCII para idiomas europeos)
  - **Unicode** (muy amplio, compatible con todos los idiomas)



Representación de la información y arquitecturas: **Sistemas de numeración**

Un ordenador solo entiende ceros y unos (0 y 1) que **representan dos estados de la corriente eléctrica**



1 → Hay corriente (estado activo o "encendido")

0 → No hay corriente (estado inactivo o "apagado")

Estos dos estados permiten al ordenador **procesar información utilizando el sistema binario**.

El concepto fundamental de codificación: convierte lo que entiende el ser humano (caracteres) en lo que entiende el ordenador (binario).

De ahí surge la necesidad de establecer un código común (como ASCII o Unicode).

Representación de la información y arquitecturas: **Sistemas de numeración**

Una vez establecido la **codificación** con el que nos comunicamos con el ordenador, es fundamental entender **qué es un sistema de numeración.**

No posicionales: el valor no depende de la posición que ocupa (ej: números romanos)

Un **sistema de numeración** es un conjunto de reglas y símbolos que utilizamos para **representar cantidades o números**.

En resumen, la **forma en que escribimos y entendemos los números**.

Existen dos sistemas de numeración

Posicionales: el valor de la cifra sí depende de la posición que ocupa. (ej: **sistema decimal** en el número 25 el 2 equivale a la decena y el 5 a la unidad, o en el 210 el 2 equivale a la centena, **sistema binario, octal o hexadecimal**)

Los sistemas posicionales son útiles para los ordenadores porque permiten representar gran cantidad de valores con pocos símbolos.

Representación de la información y arquitecturas: Teorema fundamental de la numeración

El **teorema fundamental de la numeración (TFN)** es la base del funcionamiento de **todos los sistemas posicionales**, incluidos sistema binario (base 2), el sistema octal (base 8) y el sistema hexadecimal (base 16).

Establece que el valor total de un número será la suma de cada dígito multiplicado por la potencia de la base correspondiente a la posición que ocupa el número.

Por ejemplo:

En base 10:

el número **1492 equivale a** $1\times10^3 + 4\times10^2 + 9\times10^1 + 2\times10^0 = 1492$

$1000 + 400 + 90 + 2 = 1492$

porque trabajamos en base 10, y el valor total de este número será 10.

Representación de la información y arquitecturas
Teorema fundamental de la numeración: Sistema binario

En el **sistema binario** se utilizan 2 símbolos el **0 y el 1** y se denominan **bits.**

- Al tratarse de un sistema de **numeración posicional**, los valores de 0 o 1 están condicionados por la posición que ocupan.

Si aplicamos el **TFN** al número 100101010 en base 2 obtendremos que su valor es de 298.

TFN binario → decimal

Posiciones: 8 7 6 5 4 3 2 1 0
Número: **1 0 0 1 0 1 0 1 0**
Potencias: 2^8 2^7 2^6 2^5 2^4 2^3 2^2 2^1 2^0

$1\times2^8 + 1\times2^5 + 1\times2^3 + 1\times2^1$
256 + 32 + 8 + 2 = 298

decimal → binario

Para pasar de un número decimal, por ejemplo, 298 a binario, se debe **dividir entre 2 hasta que no se pueda más y recoger el residuo en orden inverso: 100101010**

298 entre 2 sobra 0
149 entre 2 sobra 1
74 entre 2 sobra 0
37 entre 2 sobra 1
18 entre 2 sobra 0
9 entre 2 sobra 1
4 entre 2 sobra 0
2 entre 2 sobra 0
1 entre 2 sobra 1

Representación de la información y arquitecturas: Teorema fundamental de la numeración: Sistema octal

El sistema **octal (base 8)** se usa en informática cuando el número de **bits a representar es múltiplo de 3,** porque **cada dígito octal representa exactamente 3 bits** del sistema binario (ya que **$2^3 = 8$**)

- El sistema **octal tiene 8 símbolos y se representa del 0 al 7.**

Así, se puede escribir código binario largo de manera más corta, legible y se aprovecha mejor la capacidad de almacenaje.

Para saber el valor decimal al que corresponde el número **452** en base 8, utilizaremos el **teorema fundamental de la numeración (TFN)**:

TFN octal → decimal

Posiciones: 2 1 0
Número: 4 5 2
Potencias: 8^2 8^1 8^0

$4\times8^2 + 5\times8^1 + 2\times8^0$

$256 + 40 + 2 = 298$

Transformar de **binario → octal**

La conversión es directa, se tiene que agrupar los números binarios de 3 en 3 empezando por el final.

Por ej. el número binario **100101010** a octal:

100	101	010
$1x2^2$	**$1x2^2+1x2^0$**	**$1x2^1$**
4	**5**	**2**

= 452

Representación de la información y arquitecturas: Teorema fundamental de la numeración: Sistema hexadecimal

El sistema **hexadecimal** usa 16 símbolos: del **0 al 9** y de la **A a la F** (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F) y donde cada carácter tendrá el valor comprendido entre el **0 y 15.**

- 0 = 0, 1 = 1 …. 9 = 9, A = 10, B = 11... F = 15
- Cada grupo de **4 bits** se convierte en 1 cifra hexadecimal

Volvemos a utilizar el **Teorema fundamental de la numeración (TFN)** para calcular 12A en base 16:

TFN hexadecimal → decimal

Posiciones: 2 1 0
Número: 1 2 A
Potencias: 16^2 16^1 16^0

$1\times16^2 + 2\times16^1 + A\times16^0$

256 + 32 + 10 = 298

Transformar de **binario → hexadecimal**
La conversión es directa, se tiene que agrupar los números binarios de 4 en 4 empezando por el final.
Por ej. el número binario **100101010** a hexadecimal:

0001	0010	1010
$1x2^0$	**$1x2^1$**	**$1x2^3+1x2^1$**
1	2	10
1	**2**	**A = 12A**

Representación de la información y arquitecturas: Operaciones binarias: Aritméticas y lógicas

Existen dos tipos principales de operaciones:

- **Aritméticas:** sumas, restos, multiplicaciones, etc. Son para realizar cálculos
- **Lógicas:** comparaciones entre bits (AND, OR, XOR, NOT) sirven para tomar decisiones, filtrar, comparar, etc. Son la base del funcionamiento de la CPU.

Todo lo que hace un ordenador, desde sumar hasta mover archivos, se basa en estas operaciones a bajo nivel.

Representación de la información y arquitecturas: Operaciones binarias: Aritméticas

Operaciones aritméticas

Similares a las operaciones suma, resta, multiplicación y división en base 10 pero basándonos en las siguientes peculiaridades:

OPERACIÓN		SUMA	RESTA	MULTIPLICACIÓN	DIVISIÓN
A	**B**	**A + B**	**A – B**	**A * B**	**A / B**
0	0	0	0	0	-(indefinida)*
0	1	1	1 (i falta 1)*	0	0
1	0	1	1	0	1
1	1	0 (con acarreo)*	0	1	1

*Suma: 1 + 1 produce **0 y acarreo = 1** → esto activa una nueva posición a la izquierda en binario.*
*Resta: Cuando se intenta hacer 0 − 1, se produce **un préstamo, falta 1**.También se puede restar usando la lógica de **complemento a dos.***
*División: 1 ÷ 0 es **indefinida** (no se puede dividir por 0).*

Representación de la información y arquitecturas: Operaciones binarias: Lógicas

Operaciones lógicas

- Son aquellas que usan términos lógicos para operar.
- Estas operaciones se implementan mediante circuitos electrónicos básicos y son fundamentales para el funcionamiento de los sistemas informáticos.
- Es importante destacar que la **tabla de verdad** de la operación **AND** es equivalente a la de la multiplicación en la tabla de operaciones aritméticas con bits y la operación **OR** funciona como la suma.

La tabla de la verdad de las operaciones lógicas es:

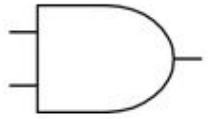
OPERACIÓN		OR	AND	XOR	NOT A
A	B	A || B	A && B	A(+)B	!A
0	0	0	0	0	1
0	1	1	0	1	1
1	0	1	0	1	0
1	1	1	1	0	0

*Una **tabla de verdad** es una herramienta que se utiliza en lógica para mostrar **todas las posibles combinaciones de entrada** de una operación lógica y cuál es su **resultado de salida**.*

Representación de la información y arquitecturas: Operaciones binarias: Puertas Lógicas

Las **puertas lógicas** son **bloques fundamentales de los circuitos digitales**.
Reciben una o más señales de entrada (0 o 1) y **generan una salida** del tipo binario (0 o 1), según una operación lógica.
Estas puertas son la base del funcionamiento de **procesadores, memorias y todo sistema informático**

AND



F = A * B

A B	F
0 0	0
0 1	0
1 0	0
1 1	1

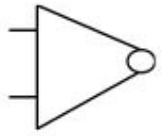
OR



F = A + B

A B	F
0 0	0
0 1	1
1 0	1
1 1	1

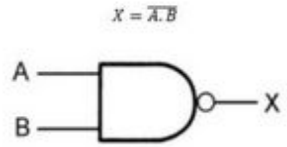
NOT



F = NOT A

A	F
0	1
1	0

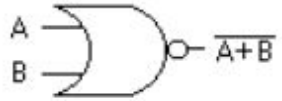
NAND



F = NOT(A * B)

A B	F
0 0	1
0 1	1
1 0	1
1 1	0

NOR



F = NOT(A + B)

A B	F
0 0	1
0 1	0
1 0	0
1 1	0

XOR



F = A (+) B

A B	F
0 0	0
0 1	1
1 0	1
1 1	0

Representación de la información y arquitecturas: Operaciones binarias: códigos de entrada/salida (E/S)

Los **códigos de entrada/salida (E/S)** son conjuntos de combinaciones de bits que asignan un valor binario a cada carácter.
De este modo, un sistema con *n* bits puede codificar hasta $\mathbf{2^n}$ caracteres distintos (por ejemplo, con 2 bits se codifican $2^2=4$ caracteres, con 4 bits $2^4 = 16$, etc.).

Representación de la información y arquitecturas: Operaciones binarias: códigos de entrada/salida (E/S)

A lo largo de la historia se han usado códigos estandarizados por fabricantes de computadores. Algunos ejemplos destacados son:

FIELDATA: código pionero de 6 (o 7) bits, desarrollado por el Ejército de EE.UU. en los años 50. Fue un estándar militar interno para UNIVAC, pero quedó obsoleto pronto.

BCD (Binary Coded Decimal Interchange Code): adaptación binaria de los códigos de tarjetas perforadas. Código de 6 bits, donde, a veces se añadía un bit adicional para verificar errores, por lo tanto, cada carácter queda identificado por 7 bits. Las cuatro posiciones de la derecha se denominan **bits de posición** (para los caracteres numéricos del 1 al 9 coincide con la representación en binario natural de estos números, por eso lo BCD se denomina decimal codificado en binario).

ASCII (American Standard Code for Information Interchange): De 7 bits (128 caracteres representables). Es uno de los códigos más usados en transmisión de datos. Con frecuencia se extiende a 8 bits (añadiendo un bit de paridad o usando extensiones) para abarcar 256 valores totales, llamado ASCII extendido.
Por ejemplo, en ASCII la letra “A” es el código decimal 65 (0100001_2)

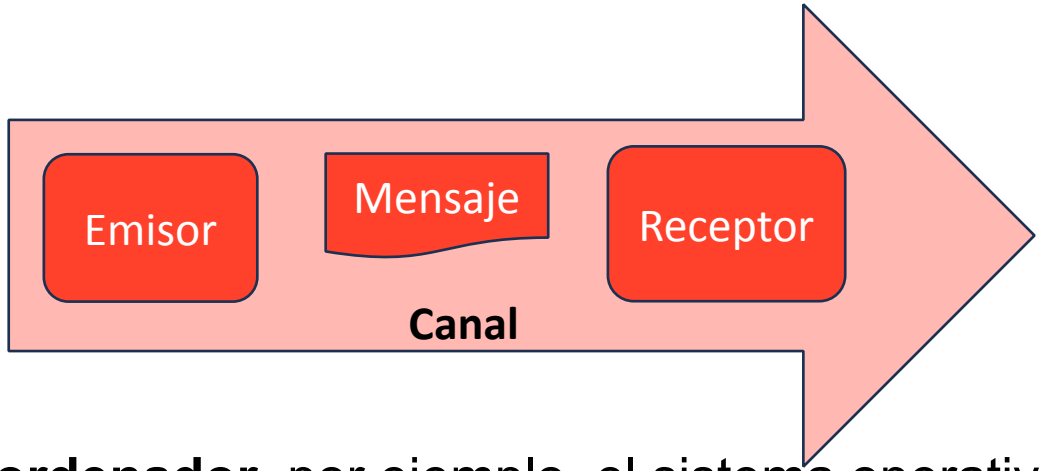
Representación de la información y arquitecturas: Operaciones binarias: códigos de entrada/salida (E/S)

ISO/IEC 8859: Usa 8 bits para codificación de alfabetos latino-occidentales.
La parte 15 (ISO-8859-15, de 1999) incluyó caracteres necesarios en idiomas europeos, como la “ç” francesa, la “ñ” española, la ß alemana, y el símbolo del euro (€).

En la **actualidad** la mayoría de los sistemas han adoptado **Unicode (p.ej. UTF-8)** que puede representar todos los caracteres de todos los idiomas, pero los códigos anteriores siguen siendo la base histórica de la codificación de texto.

Comunicación

La **comunicación** es el intercambio de información entre un ***emisor*** y un ***receptor*** a través de un *canal*. El emisor envía un mensaje y el receptor lo recibe; el canal es el medio físico o lógico por el que viaja esa información (cable, fibra óptica, aire, etc.).



En un sistema **usuario-ordenador**, por ejemplo, el sistema operativo recibe comandos del usuario (emisor) y los interpreta según su arquitectura, y luego el ordenador (actuando como emisor) entrega resultados al usuario (receptor). El equipo también puede almacenar o retransmitir los resultados mediante adaptadores adecuados (interfaces, puertos, tarjetas).

Comunicación
Tipos de comunicación

Tipos de comunicación según participantes: Se distinguen varios escenarios dependiendo de quién envía y recibe la información:

- **Usuario–Computadora:** interacción directa con la máquina (por ejemplo, teclado–PC, pantalla–usuario).
- **Computadora–Computadora:** comunicación entre dos ordenadores (p.ej. redes).
- **Computadora–Periférico:** intercambio entre el ordenador y un dispositivo (impresora, disco, sensor, etc.).

Según el canal físico:

- ***Comunicación por línea***: usa un medio físico conductor (cables de cobre, fibra óptica) para transmitir señales.
- ***Comunicación por radio***: el canal es el aire o el espacio físico sin conductor, usando ondas electromagnéticas.

Comunicación:
Según la direccionalidad de Transmisión

Según la direccionalidad de la transmisión: hay tres modos básicos de enlace de datos:

- **Simplex*:*** unidireccional. Solo una de las partes puede transmitir y la otra solo recibir.
- *Por ejemplo*, un teclado envía datos (emistor) hacia la pantalla (receptor), el teclado solo introduce datos y el monitor solo los recibe y muestra (sin volver a enviar información).
  Semi-dúplex: bidireccional alternado. Ambos pueden enviar y recibir, pero no a la vez. Un dispositivo habla y el otro solo escucha, alternando el control del canal.
- *Por ejemplo,* los walkie-talkies, donde se apreta un botón para hablar y luego se suelta para escuchar.
  Full-Dúplex: bidireccional simultáneo. Ambos pueden transmitir y recibir al mismo tiempo. *Por ejemplo,* una llamada telefónica donde las dos personas hablan y escuchan simultáneamente.

Comunicación
Según la sincronización y la naturaleza de la señal

Según la sincronización:

- **Síncrona***:* la comunicación está coordinada por relojes o pulsos de sincronismo constantes. Emisor y receptor permanecen en sintonía continuamente
- **Asíncrona***:* no hay sincronía permanente; El emisor y el receptor solo están en sincronía mientras se transmite el mensaje.

Según la naturaleza de la señal:

- **Analógica***:* la señal varia de forma continua en el tiempo y amplitud. *Por ejemplo*, las ondas de radio, de sonido o video analógicas cambian continuamente.
- **Digital***:* la señal toma valores discretos y acotados en tiempo, forma y valor, *por ejemplo*, niveles binarios 0/1.

Arquitectura

Un **ordenador digital** es un sistema complejo que combina *hardware* (componentes físicos) y *software* (programas e instrucciones).

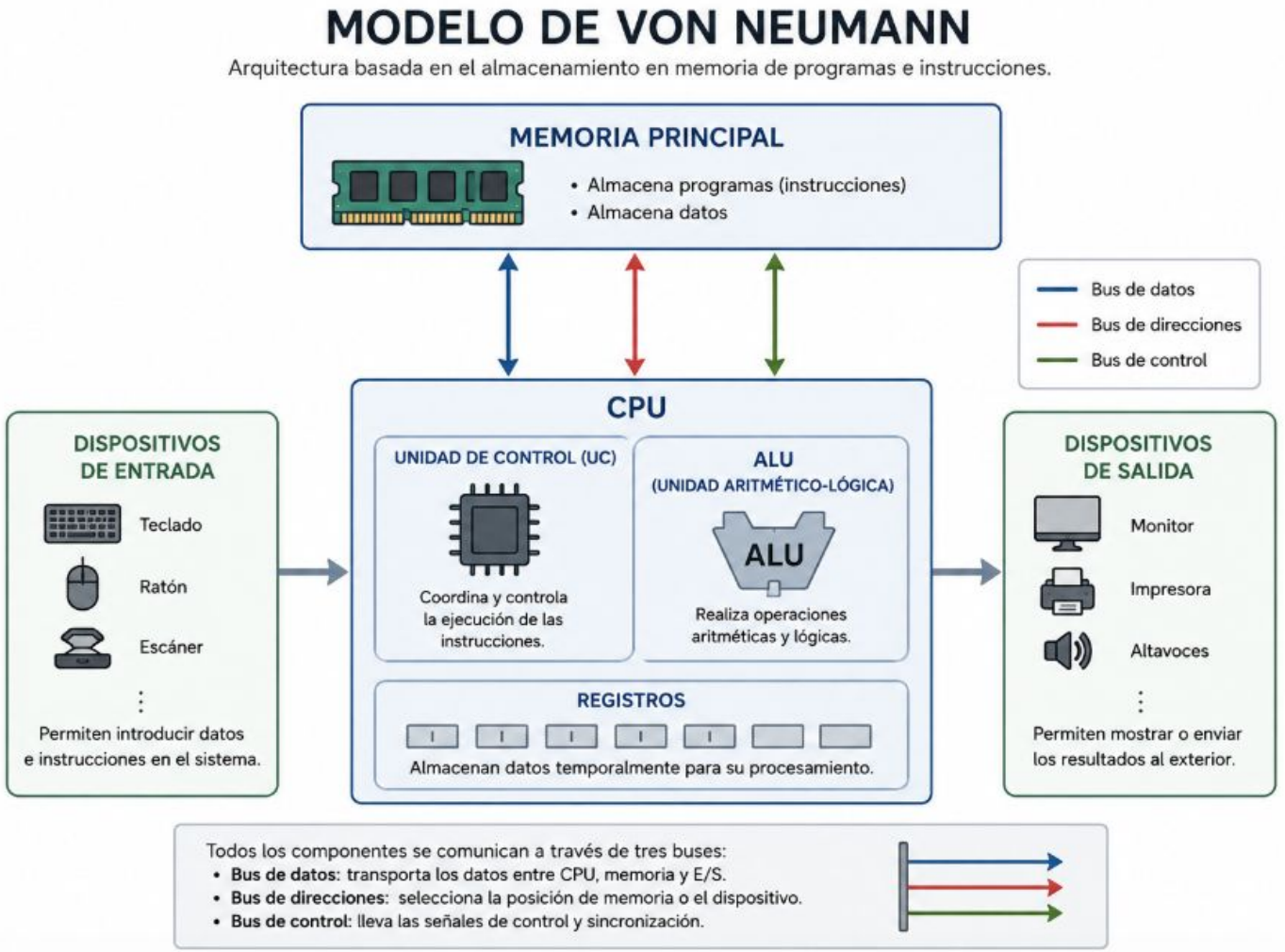
Su función principal es ejecutar programas.

- El **hardware** incluye procesadores, memoria, buses, discos, etc., es decir los elementos tangibles.
- El **software** es el conjunto de instrucciones y programas que *ordenan* al hardware cómo realizar tareas específicas.

Arquitectura: John Von Neumann

La mayoría de los ordenadores modernos sigue la **arquitectura Von Neumann**, basada en la descripción que John von Neumann (1903-1957) elaboró en 1945, que se basa en **tres ideas principales.**

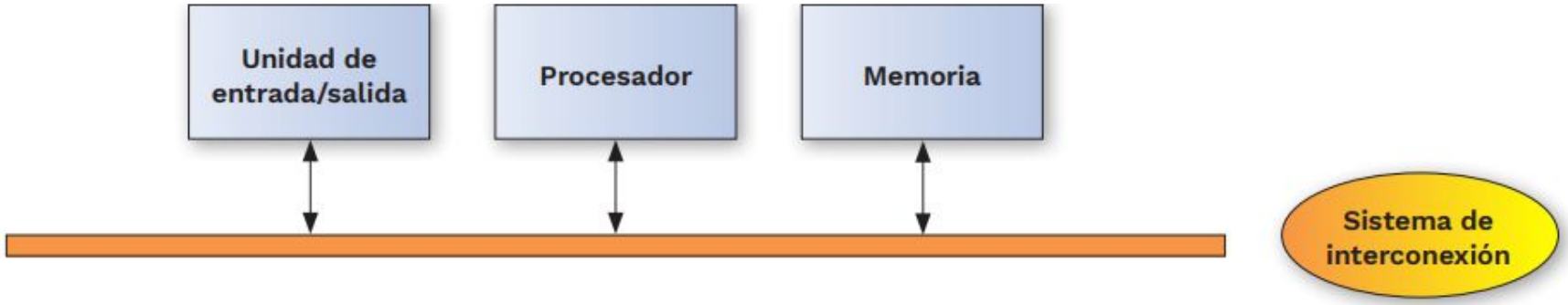
1. Memoria única para **datos e instrucciones**: ambas coexisten en la misma memoria compartida, lo que permite almacenar los programas junto con sus datos (concepto de *programa almacenado*).
2. Acceso a memoria por **dirección**: cada dato o instrucción se localiza especificando su dirección en la memoria.
3. Ejecución **secuencial**: la CPU procesa las instrucciones de programa una tras otra en orden. Cada instrucción se lee de memoria, se ejecuta, y luego se avanza a la siguiente.



Arquitectura: John Von Neumann

La arquitectura propuesta por **John von Neumann**, se compone por los siguientes elementos funcionales:

1. **Sistema de entrada/salida (E/S):** dispositivos y adaptadores que permiten leer o escribir datos fuera del computador (teclados, monitores, discos, interfaces de red, etc.).
3. **Memoria:** almacena datos e instrucciones de programa.
2. **CPU (Unidad Central de Procesamiento):** incluye la **ALU** (unidad aritmético-lógica) que realiza cálculos y operaciones lógicas, y la **unidad de control** que coordina la ejecución de las instrucciones.
4. **Buses internos:** canales de comunicación (líneas de datos, direcciones y control) que interconectan CPU, memoria y E/S.

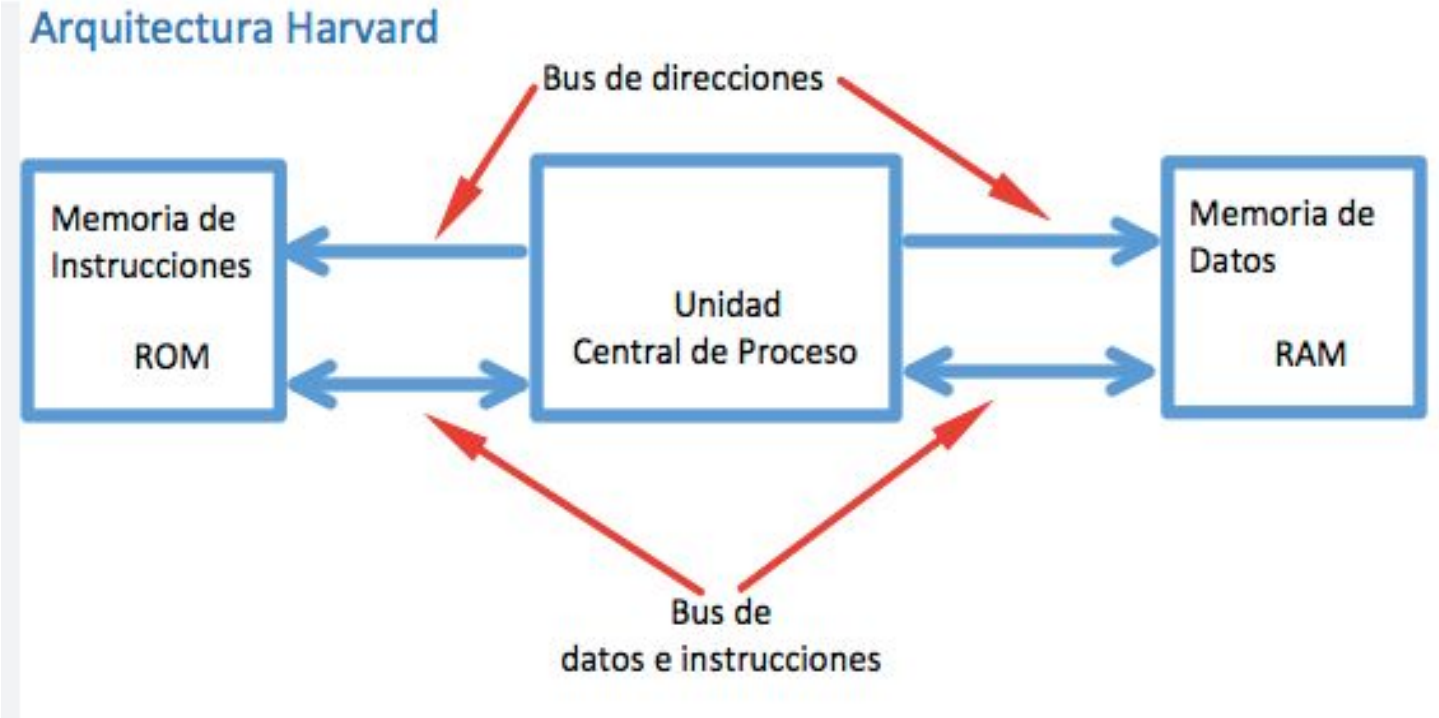


La ejecución de programas completos se logra leyendo las instrucciones desde la memoria y procesando sus datos (todo con los mismos buse).

Arquitectura: Otras arquitecturas

Otras arquitecturas: Aunque Von Neumann es la más común, existen modelos alternativos útiles en ciertos contextos:

- **Arquitectura Harvard:** separa físicamente la memoria de programa (instrucciones) de la memoria de datos. Es decir, tiene buses y espacios independientes para instrucciones y datos (en oposición al "bus único" de Von Neumann). Esto permite que la CPU lea una instrucción y un dato simultáneamente (por caminos diferentes), aumentando el rendimiento.



Arquitectura: Otras arquitecturas

- **Arquitecturas paralelas:** apuntan a ejecutar múltiples tareas en un mismo instante (concurrencia).
  - Puede darse de dos formas:
    - ***Implícita****:* el hardware/compilador gestiona la paralelización sin intervención del usuario (por ejemplo, procesadores multinúcleo que ejecutan hilos en paralelo de forma transparente)
      Si tu portátil tiene 8 núcleos: cuando abres Chrome, Spotify y Word a la vez, el sistema operativo reparte esas tareas entre núcleos sin que tú hagas nada
    - ***Explícita****:* el programador o usuario aprovecha conscientemente los recursos paralelos (p.ej. programación con hilos o procesos en ejecución simultánea). Cuando un programador escribe código que lanza varios hilos a propósito, eso es explícita.

Ampliación: Complemento a dos

Ejemplo de resta con complemento a dos de 3-5:

3 = 0011

5 = 0101

Complemento a dos de 5:

- **Paso 1: invertir bits del num. 5 → 1010**
- **Paso 2: sumar 1 → 1011**

Finalmente se suma 0011 (3)

+ 1011 (−5)

Resultado: 1110 en binario → Es un número negativo en complemento a dos → equivale a −2 en decimal

Ampliación: Von Neumann

John Von Neumann (1903-1953) fue un científico, físico y matemático estadounidense, judío de origen húngaro, considerado por muchos como uno de los más importantes científicos del siglo XX.

El año 1945 desarrolla el concepto de programa interno y describe la base teórica de construcción de un ordenador electrónico.